

Follow the steps

<https://tigp-experimental-methods.github.io/preparation.html>

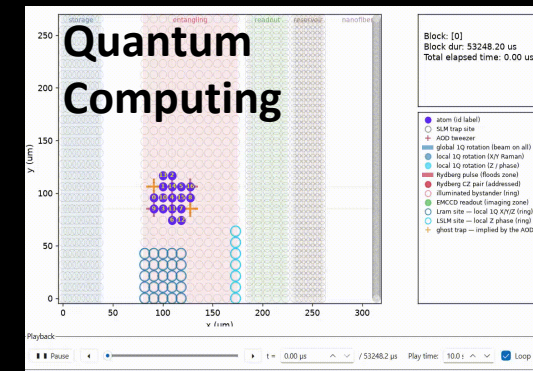
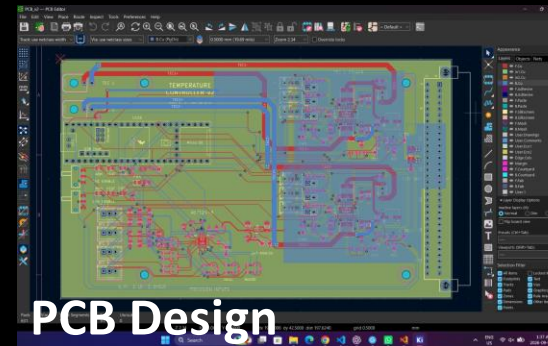
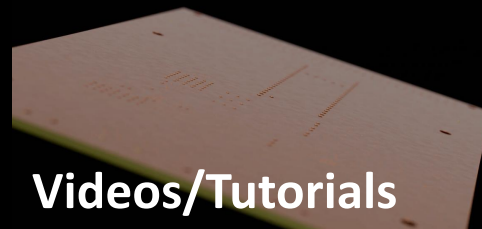
1. Make sure you have:
 - Laptop you can install software on with ≥ 10 GB space
 - USBC cable to interface to ESP32 (I have spares)
 - ESP32 board (provided)
2. Install VS Code code.visualstudio.com/download
3. VS code Claude Code extension and sign into your Claude account
4. Create a local folder for the course work and open it in VS Code
5. Let Claude prepare the rest:

Read <https://raw.githubusercontent.com/TIGP-Experimental-Methods/class-board-2026/main/SETUP.md> and follow it step by step. Ask me before installing anything, and tell me what each step is for in one sentence.

6. Create a free Cloudflare account and connect to your GitHub account

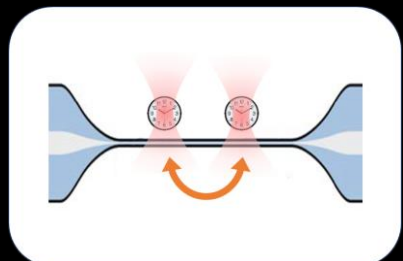
Modern Experimental Techniques

1) AI for experimentalists



Shayne Bennetts

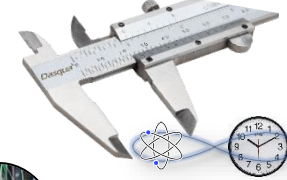
s.p.bennetts@g.iams.sinica.edu.tw



IA MS Yb Lab
Institute of Atomic and Molecular Sciences
Academia Sinica, Taiwan

<https://iamsquantum.github.io>





Our research platforms



Pathfinder

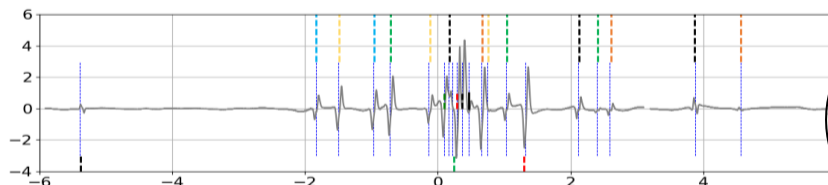
our “continuous testbed”

- Novel laser cooling
- Metastable MOT
- Continuous BEC
- Continuous Atom Lasers
- Continuous optical Lattice Clocks



Precision Spectroscopy

- Many key properties of Yb **still unknown**
- King plots can test for **new physics**

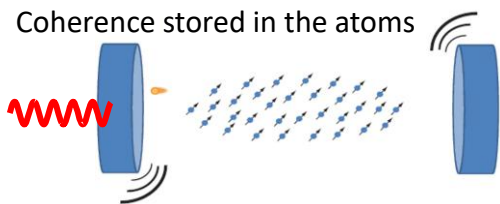


Dec 2025

Superradiant Clocks



Coherence stored in the atoms



You?

- First of its kind **new kind of optical clock**
- **High Bandwidth** precision quantum sensing

Hybrid atom-nanophotonics

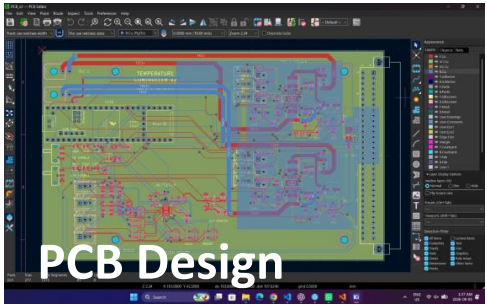
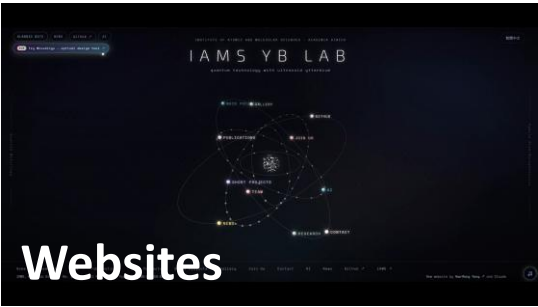
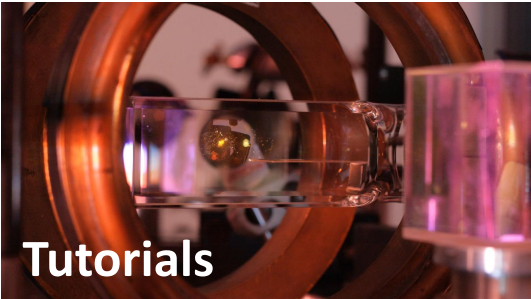
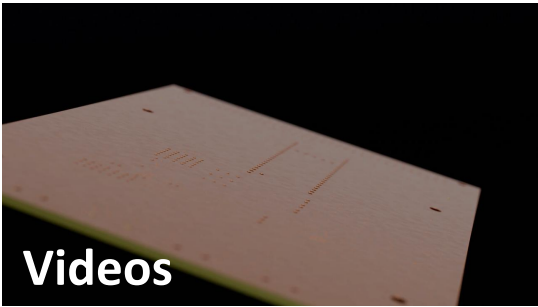
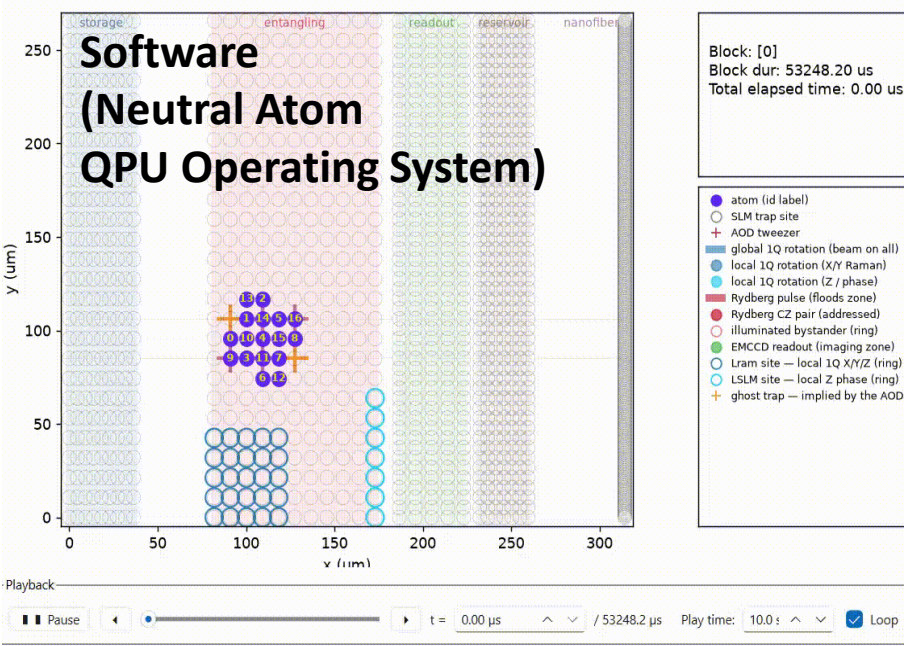
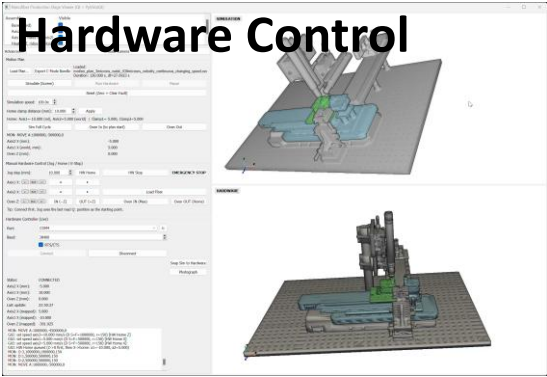
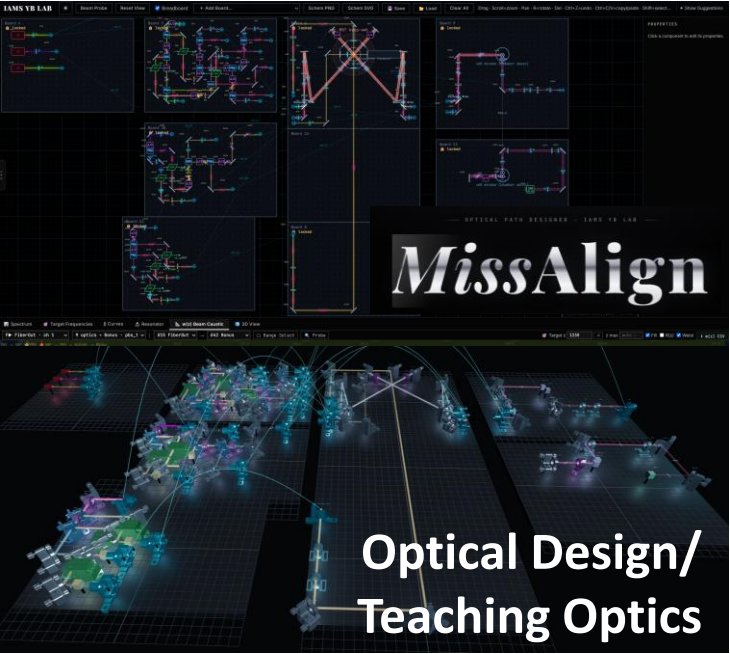
- Strong light-matter interaction
- Quantum memory
- Quantum simulation
- Subwavelength defect free arrays
- Distributed quantum computing
- Analog quantum signal processing



1st generation

2nd generation

Aim: By the end of today you will know how to use AI to help you do this



Website:	https://tigp-experimental-methods.github.io/
Workshops:	Fridays 14:20-16:20, Room 311
Language:	English (Claude can translate to mandarin)
Format:	Workshop/tutorial style - you will need your laptop (fully prepared before each class)
Claude access:	Access to Claude (Team standard) was provided to all enrolled - invitation email sent on Sunday 6th
Assessment:	Based on project: 20% Hardware demonstration 30% App/software demonstration 30% Presentation 20% Project website

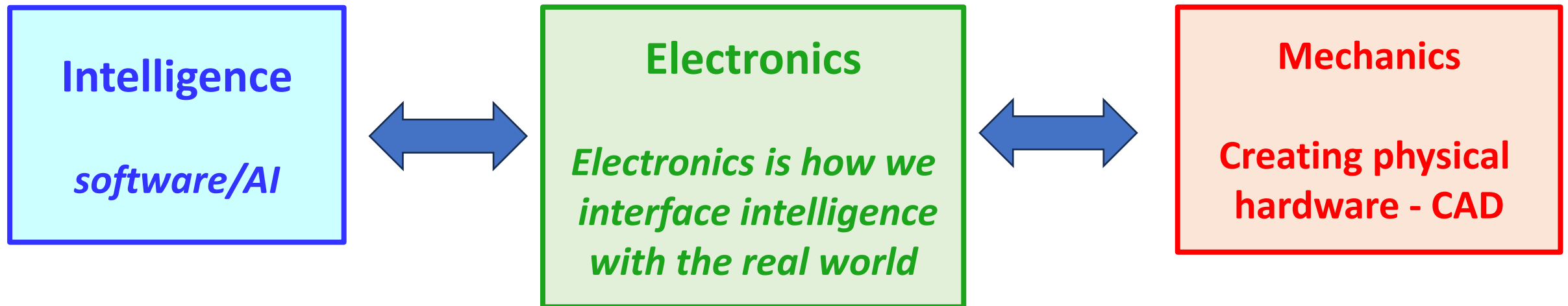
Dates:

Friday September 11	Workshop 1: AI for experimentalists
Friday September 18	Workshop 2: Designing printed circuit boards
Friday September 25	Public Holiday (no class)
Monday September 28	2pm – Deadline for cutoff for class PCB order submission
Friday October 2	Workshop 3: Firmware and basic mechanical design (PCB housing)
Friday October 9	2pm - Deadline for submission of design files for 3D printing/laser cutting
~Friday October 16	Collect printed enclosures and assembled PCBs
26-30 October	Project ~15 minute presentations/demonstration

Modern Experimental Techniques

Course Objective:

Teach key skills needed in every modern experimental physics lab.



We are not teaching how AI works or electronic/mechanical engineering (that takes years), we aim to give you a crash course in using the tools that you need to get started with your own projects.

1) Setup/Preparation: Ensure everyone has their laptop prepared, Claude access, tutor repo downloaded...

2) Introduction to AI and the tools we use

3) Project 1: Build a webapp (eg a simulator or game):

1) What do you want to make? - Suggestions: Loading a freight train game, Inverted pendulum simulator, phase locking coupled pendula, unfair golf, billiards, Calculator or simulator for something from your research?

2) Follow the “AI method” to produce a working local demo.

4) Deploy it on the web: github or cloudflare

5) Create project website to explain the concept/game in detail and links to the live app and code

6) Improve presentation: markdown -> slides -> website -> youtube video (optional extension)

**Think about what you
want to make shortly!**

4) Project 2: Interfacing with the real world

1) Aim: Create a phone app that allows you to control and measure from hardware (ESP32 – \$400NTD) over the internet (Wifi) (camera available if you want to try that too)

<https://jin-hua.com.tw/page/product/show.aspx?num=40729&kw=ESP32&lang=TW>

2) Follow the “AI method”

TiGP Workshop 2) Outline



1) Setup/Preparation: Ensure everyone has their laptop prepared and KiCAD running

2) Introduction

3) Electronics 101

How much background does everyone have?

- A crash course in electronics to understand what we are building and how it works.
- Ask your Claude tutor to explain the ideas (eg mandarin).
- Your Claude tutor may ask you questions to help teach and improve your understanding.

4) Introduction to PCB design with KiCAD, automated production and assembly

5) Project 3a: Turning our ESP32 in to a real piece of lab equipment

- Build the skeleton of our class board together
- Adding capabilities/chips to our ESP32 using KiCAD
(everyone designs different sections so as a team we can produce a powerful instrument, chips, front panel etc sections of the class board assigned (volunteers, then lots))
- Firmware/software – making our instrument useful and friendly
 - Can you adapt it to solve a problem in your current research?

Be creative!

Final PCB gerber files need to be sent for manufacturing (2pm September 28 is a hard cutoff)

1) Setup/Preparation: Ensure everyone has their laptop prepared and Onshape/another CAD program

2) Introduction

3) Mechanical fabrication techniques 101

How much background does everyone have?

- Tour of fabrication facilities @ IAMS: 3D printing FDM, resin, laser cutting, water cutter, lathe, mill

4) Mechanical design 101

How much background does everyone have?

- Using CAD - the basics: extrudes, cuts, bending, holes, threads

5) Tips for fabrication

6) Project 3b: Making a housing for our instrument

- Use Onshape (or your favourite CAD program) to draw a housing for your instrument (3D printing and/or laser cutting)
- Continue building your app/firmware/website/videos

Final housing 3D print/laser cutting files need to be manufactured (2pm October 9th is a hard cutoff)

- Collect the manufactured PCBs and housings on ~ October 16
- Assemble your ESP32, PCB, housing
- Test/refine your firmware/app

**Present Demonstrate your
project/app/website/video**

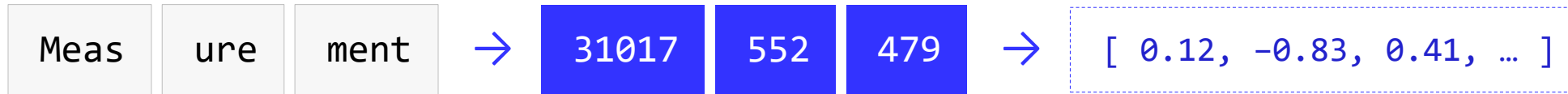
- Eg make a video to describe your app/board

A SHORT PRIMER

How large language models (LLMs) work?

Tokens, prediction, and training — the three ideas that explain almost everything else.

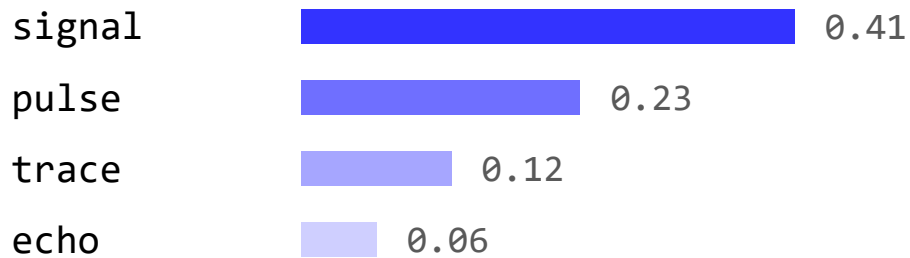
The model has no concept of letters or words. Text is split into tokens — roughly three to four characters — and each is mapped to a vector.



Each token becomes a list of a few thousand numbers. Tokens with similar meanings end up with similar lists — that geometry is the model's only handle on meaning.

Given every token so far, the model outputs a probability over the whole vocabulary.
One token is sampled, appended, and the whole thing runs again.

"The detector recorded a faint ____"



Sampling is deliberately random. Temperature controls how far down the list the model is willing to reach.

This is why the same prompt gives different answers, and why fluency is not the same thing as correctness.

TRAINING

Hide the next token in trillions of examples, measure the error, adjust billions of parameters by gradient descent.

Months of compute. Done once. Then a smaller stage tunes the model toward helpful, instruction-following behaviour.

INFERENCE

Weights are frozen. Your prompt runs forward through the network once per generated token.

Milliseconds per token. Cost scales with tokens in plus tokens out — which is why long prompts are slow and expensive.

Consequence: the model does not learn from your conversation. Nothing you type changes its weights.

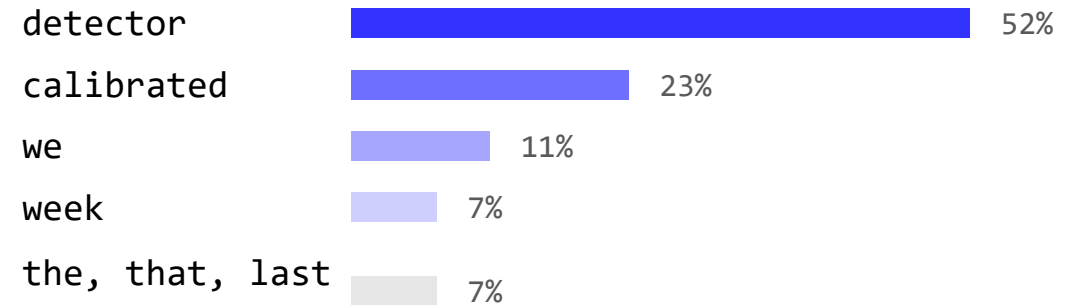
...unless your data is used for training, then the next model may know your password, your address, your research proposal and your patent.

The detector that we calibrated last week ready.

Should the blank be **is** or **are**? That depends on *detector* , six words back — not on *week*, the word right before it.

Attention is the step where the blank looks at **every earlier word** and decides how much each one matters for this prediction.
Nothing more mysterious than that.

HOW MUCH EACH WORD MATTERS TO THE BLANK



The percentages always add up to 100 — attention is a fixed budget being shared out.

EARLIER WORDS + TAGS

detector
tag: singular noun
calibrated
tag: past action
we
tag: plural pronoun
week
tag: time expression
the · that · last
tag: function words

TAG ↔ REQUEST MATCH



THE BLANK'S REQUEST

"I'm about to be a verb. Find me my subject — a singular noun."

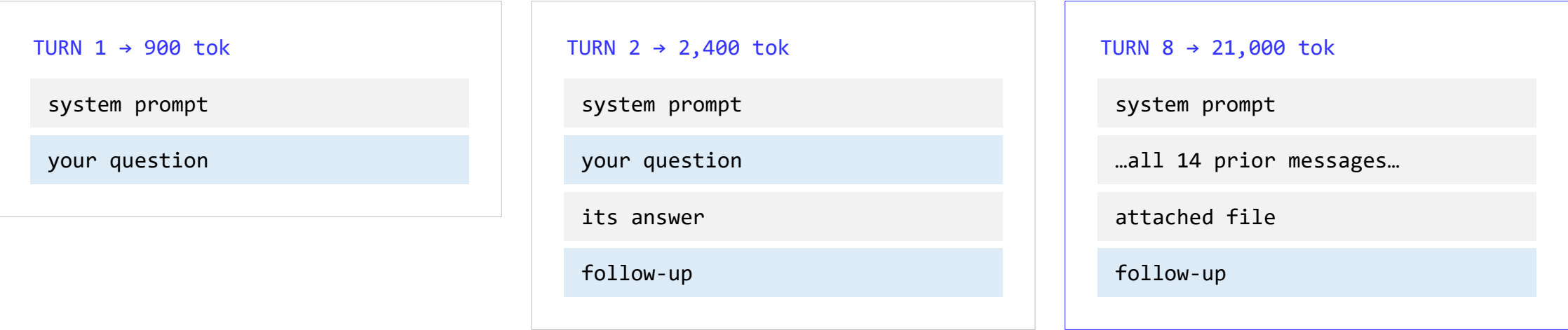
Result: copy each word's contents in those proportions. The blank now knows its subject is singular → **is**.

jargon — the tag is the **key**, the request is the **query**, the contents copied are the **value**.

Runs for every word at once, many searches in parallel, each looking for a different kind of thing; repeated layer after layer.

Every word checks every other word, so doubling the text roughly quadruples the work.

The model keeps no memory between calls. What feels like a session is the client re-sending the full transcript each time, so the model can attend to it.



So a long chat gets steadily slower and more expensive per message, even when your last question was one line.

Attention keys and values for a prefix can be stored and reused. A cache hit skips that computation — typically far cheaper and noticeably faster than processing those tokens again.

Turn 2	system + docs + history – cached, reused	new question
Turn 3	system – cached	edited an earlier message → everything after it recomputed

The match must be an exact prefix. Change one character near the top and the cache below it is void.

Practical order: stable instructions and documents first, volatile turn-by-turn text last.

Because the whole transcript is resent and attended over, off-topic history is not neutral — it is paid for and it competes for the model's attention.

DILUTED ATTENTION

Attention weight is normalised across all tokens present. Unrelated earlier text pulls probability away from what you actually asked about.

STALE CONSTRAINTS

Instructions from the earlier topic — a format, a dataset, a naming convention — remain visible and get applied to the new one.

COST AND SPEED

You re-pay for every token of the old topic on every new message, and the quadratic cost of attention makes it slower too.

New topic, new chat. If you need something from the old one, paste just that part across.

The file is converted to tokens and placed at the start of the context. From then on it is resent with every message and attended over by every word the model produces.

ROUGH TOKEN COST

one question		~50
one photo or plot		~1,500
20-page paper, text		~15,000
same paper as page images		~30,000+

Orders of magnitude, not exact figures — they vary by model and file.

PAID ON EVERY TURN

Those tokens ride along with each new message for the rest of the session — slower and costlier from turn one.

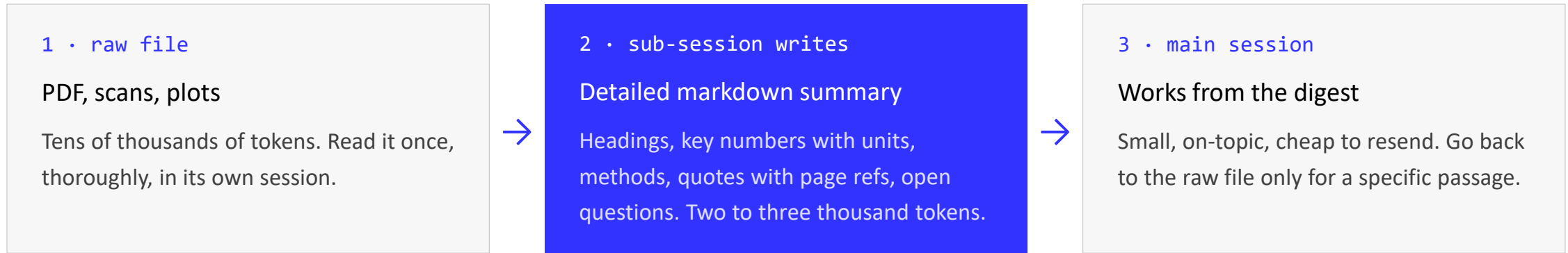
MOSTLY NOISE

For any one question, most of a paper is irrelevant — yet every word must still divide its attention budget across it.

LOSSY FOR IMAGES

Scanned pages, tables and plots become tokens once, imperfectly. Small text and axis labels are where reads go wrong.

Pay the cost of the raw file once, in a disposable session. Hand the main session a compact, structured markdown digest instead.



Ask for detail, not brevity. A digest that drops the numbers forces you back to the file. Tell the sub-session what the main task will need.

Text is the native format. Markdown tokenises cleanly and is attended over exactly; page images are neither.

Each tier is the same architecture at a different scale. Bigger means better judgement on hard, ambiguous work — and proportionally slower and more expensive on every token, in and out.

Haiku 4.5 \$1 / \$5 1× Fast, cheap, high volume Classification, extraction, summaries, first-pass drafts, tests that already have a spec.	Sonnet 5 \$3 / \$15 3× The everyday default Most writing, analysis and coding. Near-Opus quality on routine work at a third of the cost.	Opus 5 \$5 / \$25 5× Hard reasoning, long agentic runs Architecture decisions, multi-file changes, anything where an early mistake compounds.	Fable 5.1 \$10 / \$50 10× Frontier, for the hardest problems Days-long autonomous work, root-cause debugging, research-grade reasoning.
---	--	---	---

Open AI Equivalent:

Luna

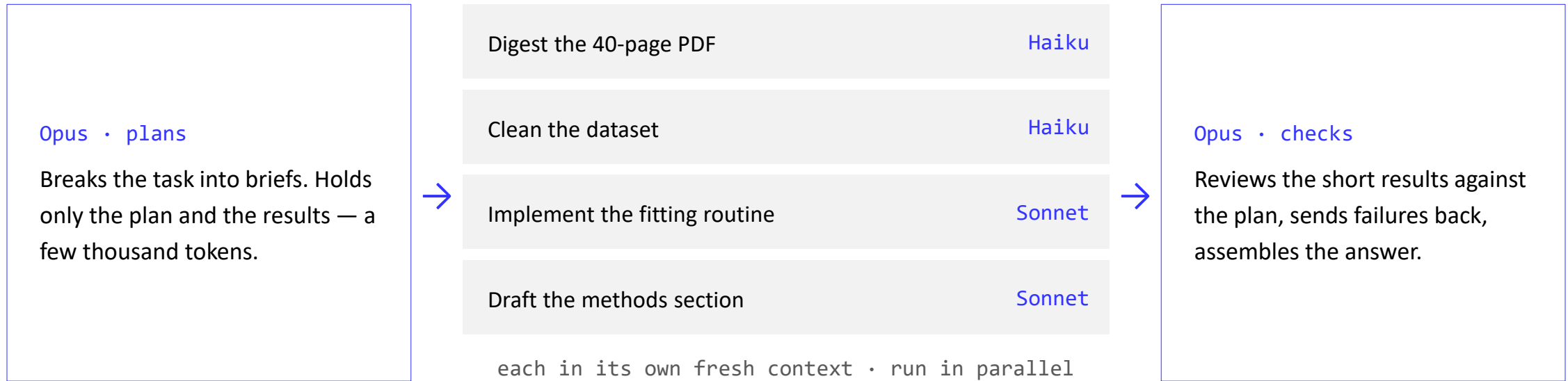
Tera

Sol

Astra

Chinese origin open source models Kimi, DeepSeek etc can be cheaper per token but are less efficient/weaker
They are also a security threat and are **not allowed**.

Delegate each bulk step to a subagent on a smaller model with its own fresh context. It returns a short result; the raw material never enters the planner's window.



Cost. The bulk tokens run at 1–3× instead of 5–10×. The expensive model reads briefs and results, not raw files.

Context. The planner's window stays small and on-topic — the same rule as one topic per session, applied to a whole project.

1) Vision:

Describe what you want, with as much detail as possible,

2) Specification/Plan:

Get AI to produce a detailed specification - plan the project!

Break the project into many small tasks with testable deliverables

Ask AI to ask you questions!

3) Review:

Review the plan, have AI review the plan to identify improvements/flaws.

- eg Use Codex or a fresh session to review

4) Code (using agent swarms)

Tell AI to run swarms of agents in parallel to complete the build

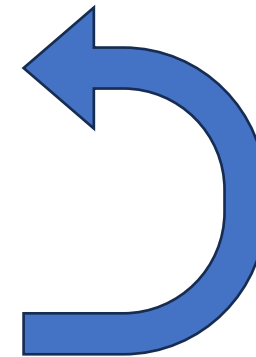
5) Test/Debug/Verify:

Ensure your plan/spec includes methods to test the code.

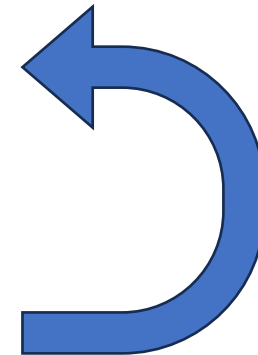
Fix problems one at a time

The most important slide in the talk!

Iterate the plan:
The first plan is *never* the last plan



Iterate



Iterate

***Never* trust AI without testing and verifying.**

A model reviewing its own output shares its context and its blind spots, and will tend to confirm it. Verification has to come from somewhere with a different view.

INDEPENDENT REVIEWER

A separate agent that sees only the spec and the output

Fresh context, no memory of how the answer was produced. Ask it to find problems, not to approve. A stronger tier here is money well spent.

TESTS WRITTEN FIRST

Write the checks from the plan, before the work starts

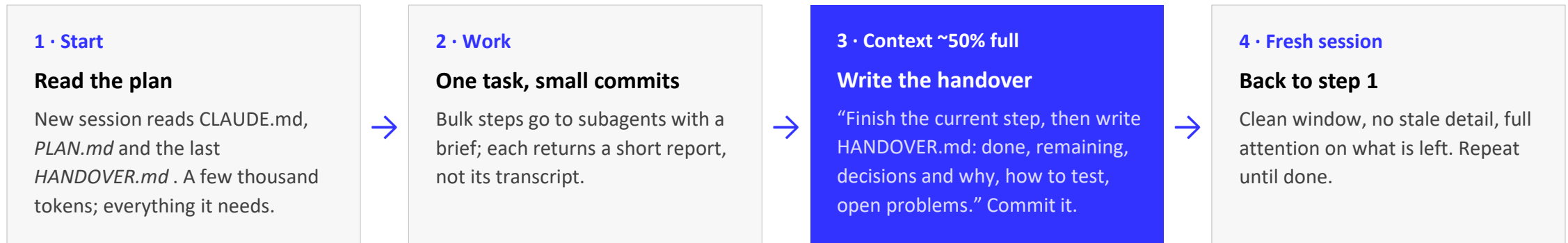
Unit tests, a known-answer dataset, a conservation law that must hold. Fixed before generation, they cannot be bent to fit the output.

Loop, don't hope. Fail → return to the worker with the failure → re-check. Most fixes land on the second pass.

Check what matters. Numbers, units, citations, code paths. Style rarely needs a reviewer.

This is the answer to hallucination. Fluent-but-wrong survives self-review; it does not survive a test it didn't write.

A session is not a project. Context fills, quality drops, and eventually the window is full of stale detail. Long work is a **chain of short sessions** joined by documents on disk.



The disk is the memory. Specs, plans, decisions, handovers, test results — all markdown, all in git. A session that can be lost without losing anything is a session set up correctly.

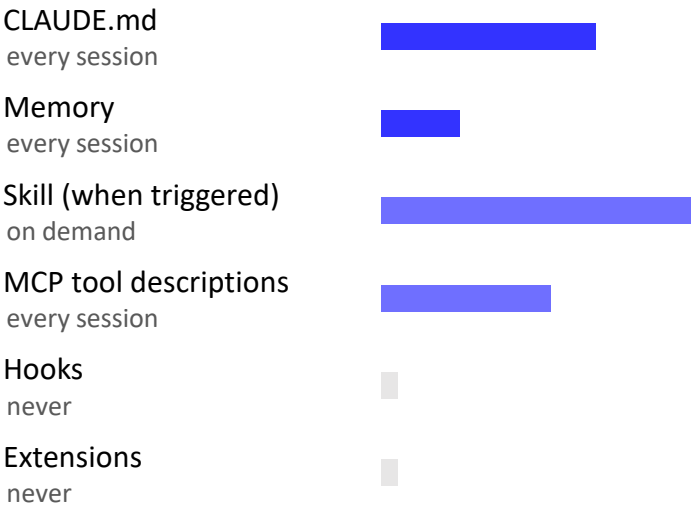
Documentation is for the next agent. Write the README and the docstrings as you go; they are what a fresh session reads instead of the code, at a tenth of the tokens.

Why 50%, not 90%. Quality degrades before the window is full — attention spreads over stale detail. Hand over while the agent can still write a good summary.

An agent starts every session knowing nothing about you. These are the six places you put what it should know — and each one costs context differently.

CLAUDE.md	A markdown file at the project root, read at the start of <i>every</i> session. Standing rules: how to run tests, naming conventions, what never to touch, where the docs are. Keep it under a page.
Memory	Notes the agent writes for <i>itself</i> and re-reads next session: “this repo uses conda env <i>lab</i> ”, “user prefers plots in SI units”. Persistent across sessions; small; auto-loaded. It is not the conversation — a new session still starts empty apart from this.
Skills	Instructions + scripts for one kind of task (make a video, write a report). Loaded only when the task matches, so many skills cost nothing until used. <i>show-your-work</i> is five of these.
MCP servers	Model Context Protocol: a standard plug for giving the agent <i>tools</i> — read a Google Sheet, query a database, drive the browser, control an instrument. Each tool’s description sits in context; the tool itself runs outside.
Hooks	Shell commands that run automatically on events — after every file edit, before a commit, at session end. Run the tests, run the linter, block a dangerous command. Zero tokens; the model cannot skip them.
Extensions	The VS Code layer: Claude Code, Codex, Copilot, Python, Jupyter, Git graph. Which agent you use is just another extension.

WHAT EACH COSTS IN THE WINDOW



Everything marked “every session” is re-sent on every turn. A bloated CLAUDE.md, a long memory file or twenty MCP servers can eat a quarter of the window before you type a word.

Prefer hooks over instructions for anything that must always happen:

“run pytest after edits” as a hook is free and guaranteed; as a sentence in CLAUDE.md it costs tokens and is sometimes forgotten.

Put all needed data in the prompt	The model attends only to what is in the window. Paste the data rather than hoping it was memorised.
One topic per session	The whole transcript is resent and competes for attention. New topic, new chat.
Summarise files first	Let a disposable session turn PDFs and images into a detailed markdown summary; work from that.
Use cheap workers, strong planner	Route bulk steps to smaller models in fresh contexts; keep the strong model for planning and review.
Verify independently	Numbers, citations and code get checked by a separate reviewer or by tests written before the work.

A SHORT PRIMER

Failure modes

Like people AI makes mistakes.

1

The model is trained to produce plausible continuations, not verified ones. There is no internal fact-check step.

2

A confident tone is a property of the text distribution, not evidence of confidence in the answer.

3

Rare facts, exact numbers, and citations are where fluent guessing shows up most often.

Grounding the answer in text you supply moves the work from recall to reading — a task the mechanism is much better suited to.

INVENTS SPECIFICS

Fake citations, part numbers, constants

They read exactly like real ones — that is what a plausible continuation is. Look up every reference and every number you did not supply.

AGREES WITH YOU

A wrong premise in the question survives

Ask “why does X cause Y?” and it explains why, even if it doesn’t. State the question neutrally; ask it to argue the other side.

ERRORS ARE SMALL

Right structure, wrong sign

The shape is correct so review skims past it. Test with known answers and limiting cases, not by reading.

REQUIREMENTS DROPPED

80% done, presented as finished

Long specs lose items silently. Number the requirements; have the reviewer tick each one off against the output.

ERRORS MULTIPLY

95% per step is 36% over 20 steps

Every agentic step is a chance to drift. Verify at each step — tests, assertions, a checking agent — not only at the end.

STALE CONTEXT

Applies yesterday’s constraint to today’s task

Instructions from an earlier topic still sit in the window. New topic, new session.

Rule: use AI where checking is easier than doing.

Most experimental work qualifies — a fit, a parser, a control loop can all be tested against data you already have.

When the model can *act*, a second family of failures appears. These are caught by mechanism — hooks, git, tests — not by reading more carefully.

Gaming the test

Makes a failing test pass by weakening the test, hard-coding the expected value, or catching the exception. Protect tests in git; review test diffs first.

Scope creep

Asked to fix one function, refactors the module. Say “change nothing else”; review the diff for files you did not expect.

Wrong target

Edits a similarly-named file, or the copy in the wrong folder. Give explicit paths; keep the workspace tidy.

Claims without running

“All tests pass” with no test output. Hooks that run the tests and paste the result remove the option to claim.

Loops

Tries the same fix repeatedly with small variations. Set a step budget; after three failures, stop and ask.

Destructive actions

Deletes, force-pushes, drops a table to “start clean”. Deny-list those commands in hooks; work on a branch; commit first.

Design so the failure is visible. Tests it cannot edit, hooks it cannot skip, a git history you read. The agent does not need to be trustworthy if the system makes lying hard.

Independent review is not optional. Same context → same blind spots. The reviewer reads only the spec and the diff.

In some sense...

Hallucinations = creativity

- Not necessarily bad

Hallucinations are expected whenever the
context/specification is insufficient

Testing and validation is essential to achieve alignment
(creativity in the right places)

A SHORT PRIMER

Tools

VS Code, the Claude extension, Git

ANTHROPIC**Claude Cowork**

Give Claude a folder and a goal; it reads, edits, creates, and can drive your browser and apps. Runs in the cloud, keeps going when the laptop closes, can be scheduled. Non-coding work: files, email, spreadsheets.

ANTHROPIC**Claude Design**

Brief in, editable deck / doc / prototype out. Matches your template, exports to PowerPoint. This deck was made here. Iterate in plain language.

OPENAI**Codex app**

Same idea from OpenAI, coding-first: point it at a repo, it plans, edits, runs tests in a cloud sandbox and opens a pull request. Computer-use mode drives your desktop and browser for tasks that need real apps.

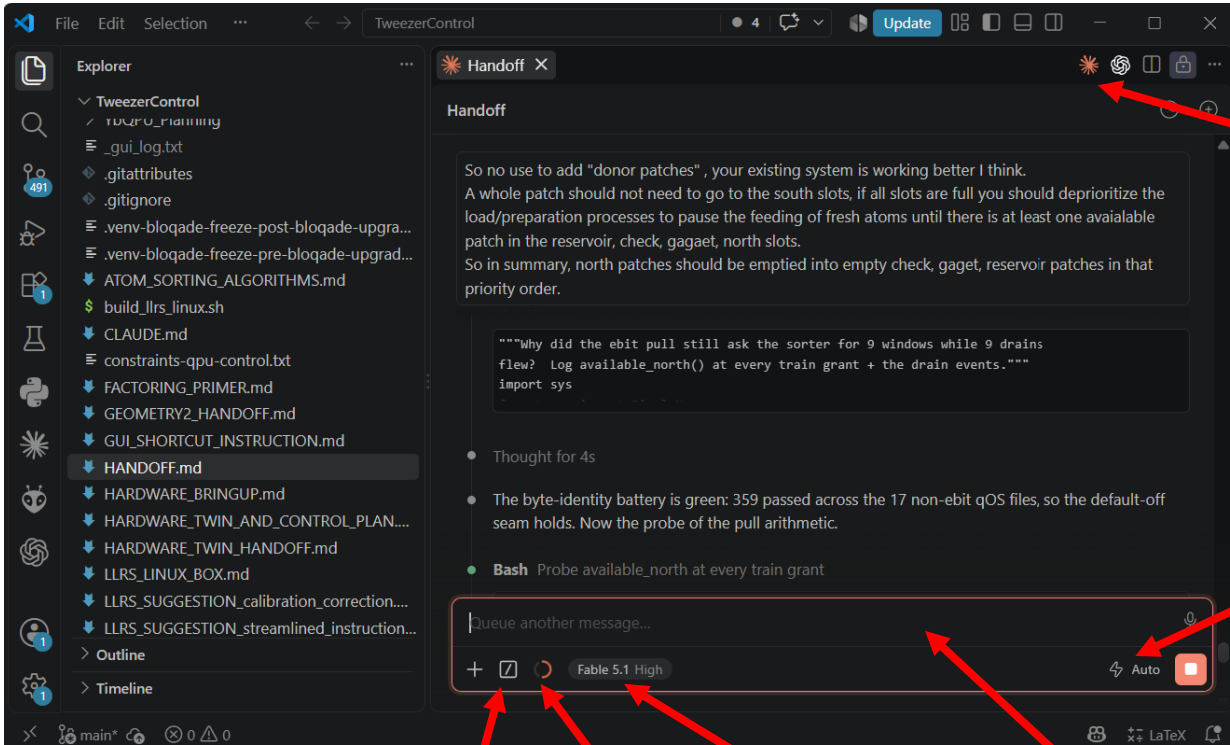
OPEN PLATFORM**VS Code + agents**

One editor, any agent: Claude Code, Codex CLI, Copilot, Gemini, a local model. The whole folder tree is context. Git, terminal, tests and the agent share one workspace.

	COWORK	DESIGN	CODEX APP	VS CODE
Best at	Computer use, files, office tasks	Slides, docs, mock-ups	Coding + computer use , PR review	Coding, large multi-file projects
Context	Folder + connectors	Your brief + template	Repo + sandbox	Entire workspace, many files
Model choice	Claude only	Claude only	OpenAI only	Any — swap per task
Lock-in	High	High	High	Low — code, config and history stay yours

Rule of thumb. Quick office task or a deck → the vendor app. Anything with a codebase, more than a few files, or that you will still be working on next year → VS Code, where the agent is a plug-in and the project is not.

...codex/claude apps both lead to “lock-in”, chat windows unsuitable for “serious” work with substantial context

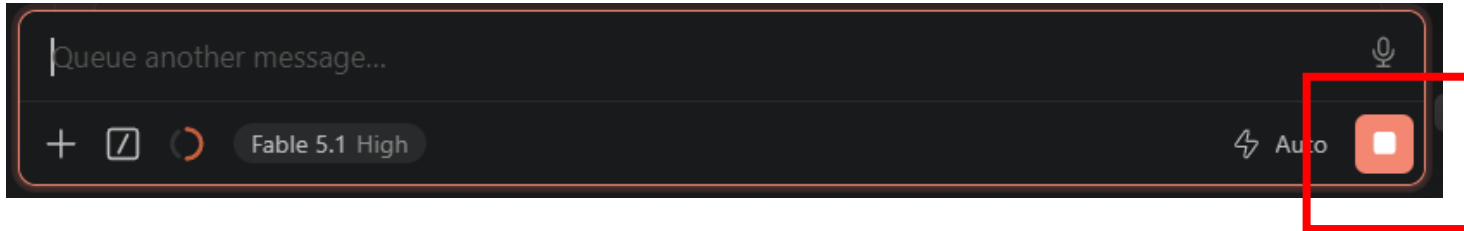


The image shows a screenshot of the Visual Studio Code (VS Code) interface with the Claude extension installed. The interface is annotated with red arrows pointing to various components:

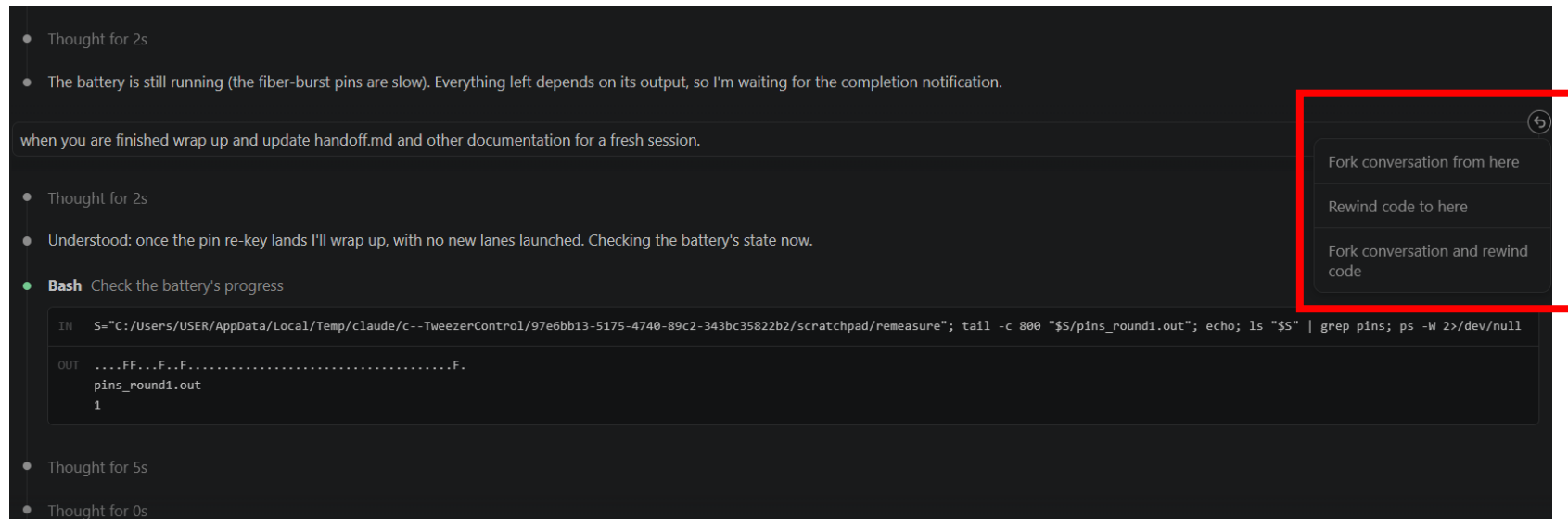
- File Explorer**: Points to the Explorer sidebar on the left.
- Git**: Points to the Git icon in the sidebar.
- Extensions**: Points to the Extensions icon in the sidebar.
- Launch a new session**: Points to the star icon in the top right of the Handoff window.
- Permissions**: Points to the permissions section in the Handoff window.
- Prompt window**: Points to the input area at the bottom of the Handoff window.
- Model/thinking level**: Points to the model selection dropdown (currently showing 'Fable 5.1 High') in the prompt window.
- Session context level**: Points to the session context menu (showing a plus sign and a checkmark) in the prompt window.
- Menu**: Points to the plus sign icon in the prompt window.

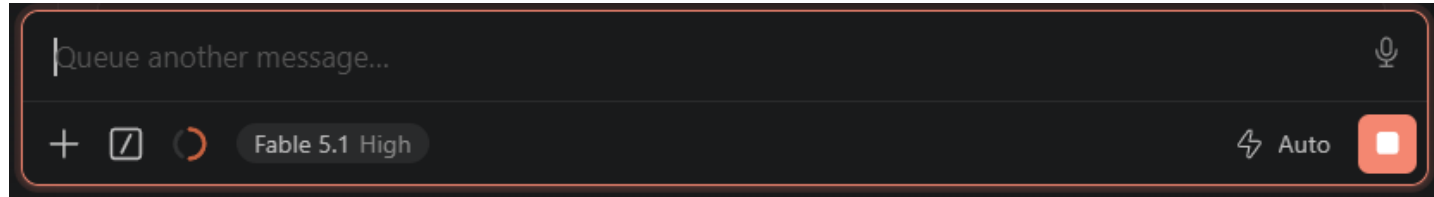
The Handoff window displays a chat session with Claude, showing a response to a query about "donor patches" and a code snippet for a function named `Log available_north()`.

1. Save your tokens: Stop the execution!



2. Don't rot/poison your context: **Rewind**

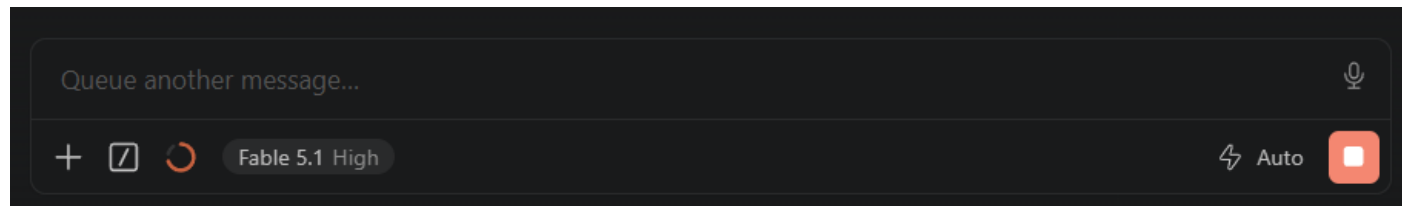




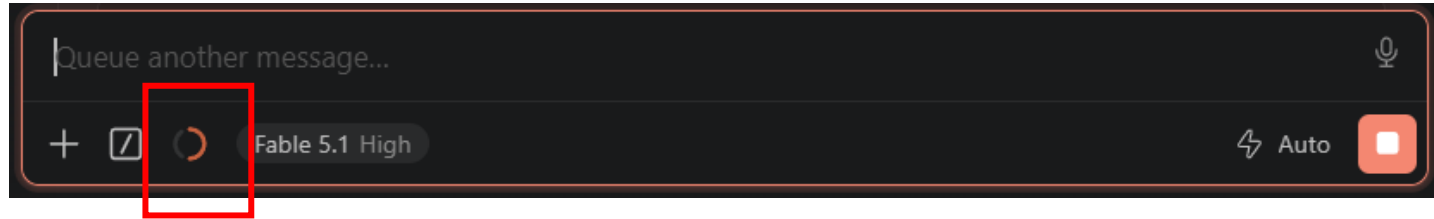
Context is full! start a new session:

Example prompt:

“Please finish what you are working on and document anything from this session that may be useful to future Claude sessions in .md files in this directory and a handover.md file to allow Claude to continue our work in a new session.”



Context is REALLY full! start a new session now!!



Clicking the context icon runs “compaction”, an automated algorithm that **deletes** 95% of the current context **without saving** useful knowledge anywhere.

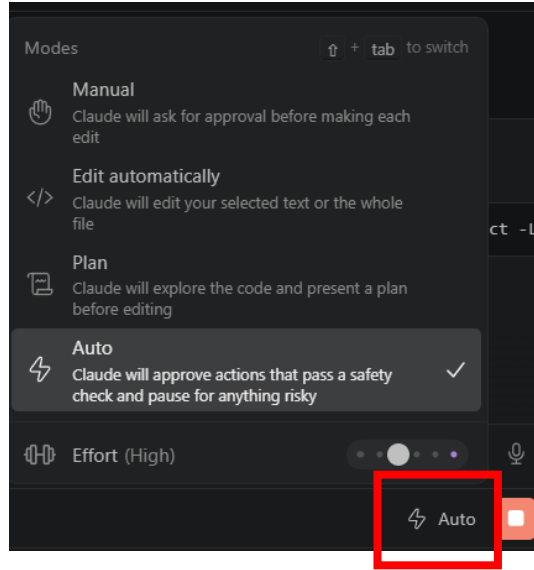
Compaction is lossy and unsuitable for long running/large tasks!

- much better to have Claude properly document the session and maintain project documentation in a hierarchical, searchable .md format!

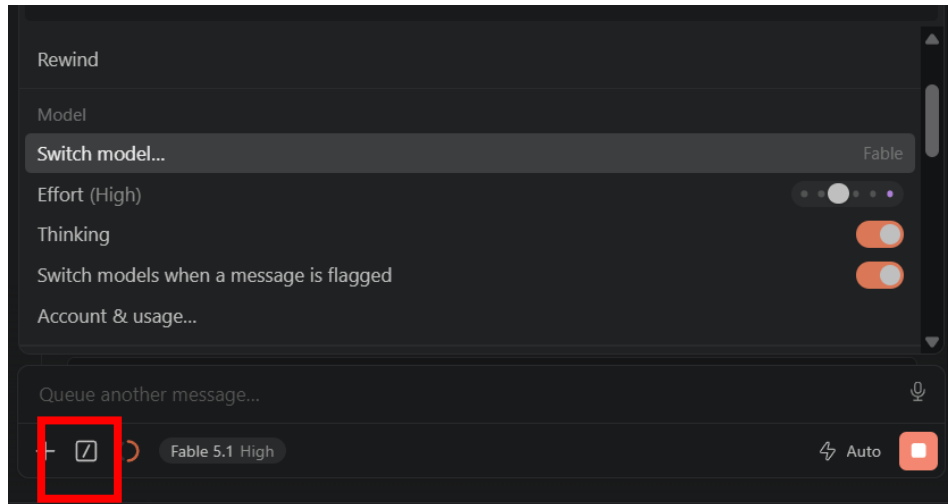
This preserves knowledge/context for large projects.

Always have Claude **write a log and handoff.md** that primes the next session with the project state.

Permissions/ Mode:

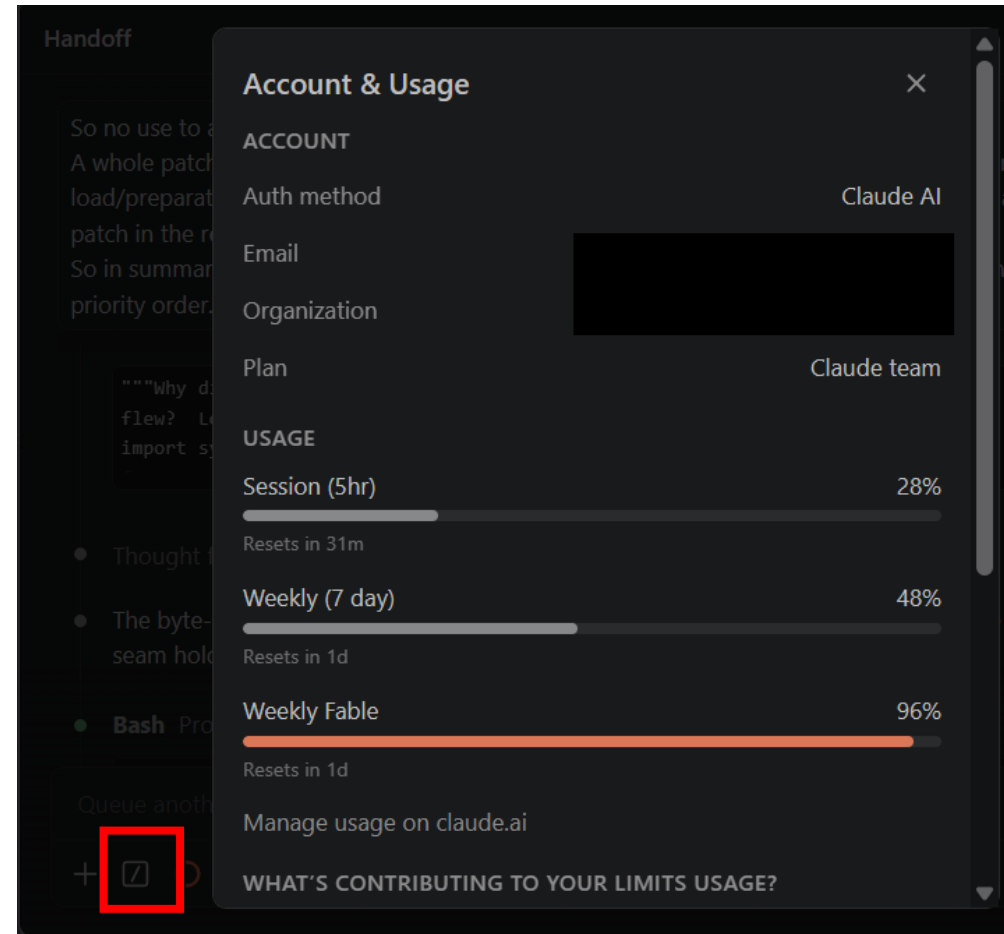


Menu: Model selection/ Thinking on (I use thinking Med or High):



Check your Usage often!

Check if unnecessary skills/hooks etc are being used on every single session cluttering context?



Every project drifts into *analysis_v2_final_FIXED.py* . Git replaces that with one folder and a complete, searchable history of every change — who, when, why, and exactly what.

SAFETY NET

Go back to any earlier state

Every saved snapshot is recoverable. Break something on Friday, restore Thursday in one click.

UNDERSTANDING

See exactly what changed

A diff shows the lines added and removed between any two points. No more “what did I touch?”

COLLABORATION

Work in parallel without overwriting

Two people — or you and an agent — edit the same project; git merges the changes rather than clobbering them.

Git is the tool; **GitHub** is a website that hosts a copy of your repository so you can share it and back it up.

We will use it entirely from **VS Code’s Source Control panel** — no terminal required.

A **repository** is a project folder that git is watching. A **commit** is a named snapshot of every file in it. The chain of commits is the **history**.

HISTORY — NEWEST AT TOP

- a41f2c Fix normalisation in fit routine
- 9b03e7 Add error bars to figure 2
- 7d8a10 Load raw data from CSV
- 1c55b9 Initial analysis script

Each dot is a full snapshot with an author, a time, and the message you wrote. Click any two to see the **diff** between them.

WHAT IS STORED

Text files: code, notebooks, LaTeX, Markdown, small data. Git stores every version compactly.

WHERE IT LIVES

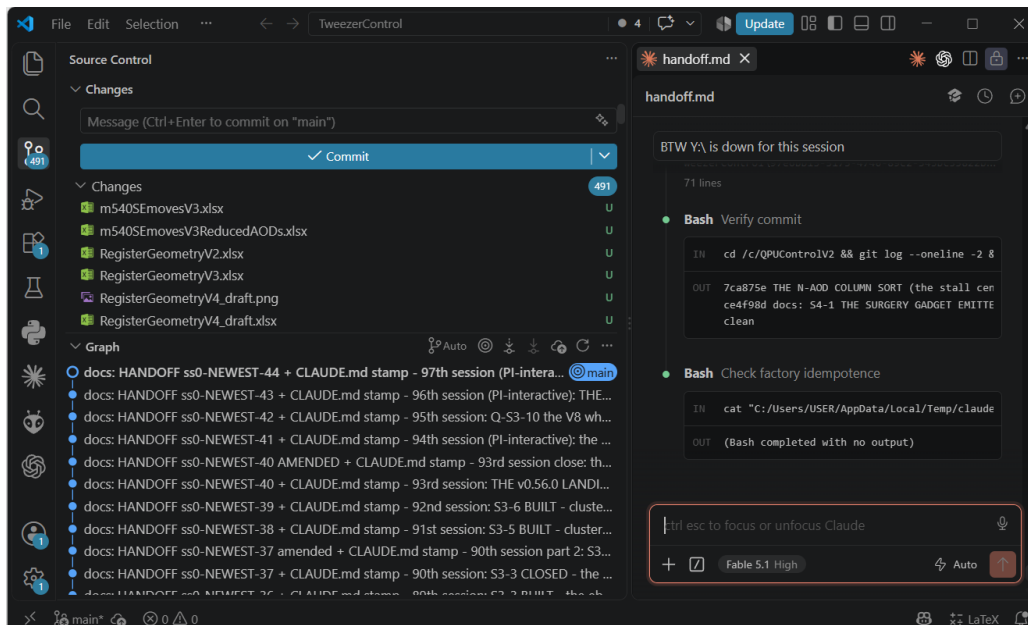
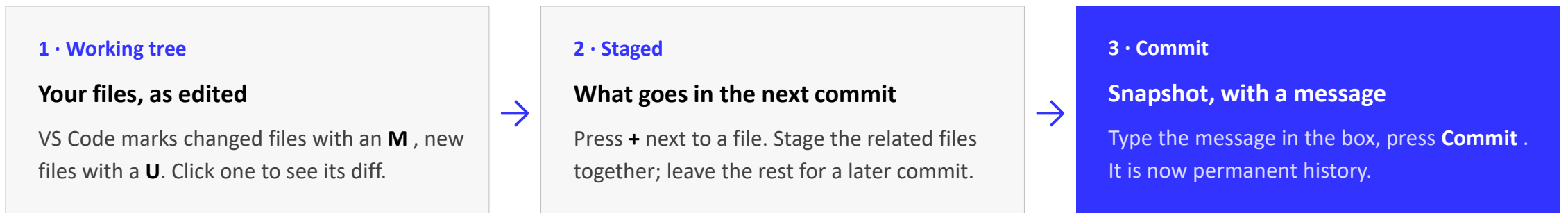
In a hidden `.git` folder inside the project. Delete that and it is just a folder again.

A GOOD MESSAGE

Says *why*, not *what* — the diff already shows what. “Fix sign error in phase term” beats “update file”.

Git: The loop: edit → stage → commit

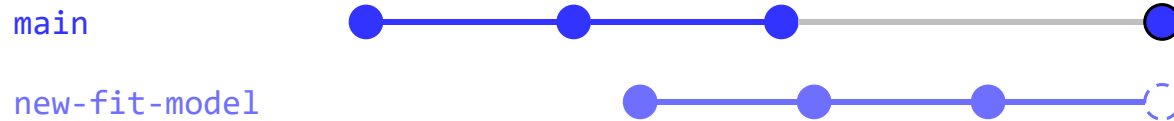
Editing does nothing to git. You choose which changed files go into the next snapshot (**stage**), write a message, and **commit**. Repeat many times a day.



Commit small and often. One idea per commit: “add fit function”, then “plot residuals”. Small commits are easy to read and easy to undo.

Commit before anything risky — a refactor, a new library, or handing the folder to an agent. That snapshot is your undo button.

A **branch** is a separate line of history. Experiment on it while **main** stays working; **merge** when it succeeds, delete it when it doesn't.



Branch off main, make three commits on *new-fit-model*, then **merge** back: the outlined commit on main contains both lines of work.

IN VS CODE

Click the branch name in the bottom-left corner → *Create new branch* . Same click switches back.

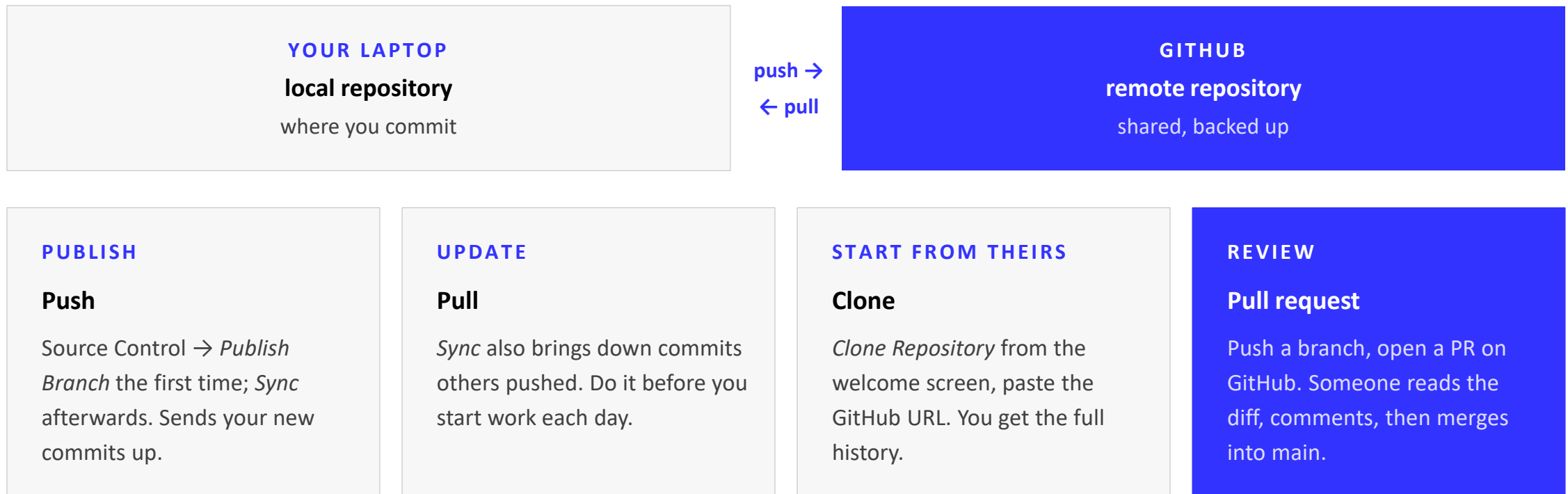
MERGING

Switch to main, Source Control menu → *Branch* → *Merge* , pick the branch. If both sides changed the same lines, VS Code shows a conflict editor: choose which to keep.

RULE OF THUMB

Main always runs. Anything unfinished lives on a branch.

Your repository is complete on your laptop. **GitHub** keeps a second copy online — for backup, for collaborators, and for review before merging.



KEEP OUT WITH .GITIGNORE

A text file listing patterns git should never track. Git is for source, not for things you can regenerate or download.

```
data/raw/ *.h5 *.npy __pycache__/.env *.aux *.log  
*.synctex.gz
```

Raw data (large, immutable), generated outputs, caches, secrets, and LaTeX build files. Keep the *script* that produces the output, not the output.

UNDOING, SAFELY

Not committed yet, want it gone

Right-click the file in Source Control → *Discard Changes* . Back to the last commit.

Committed, want to undo it

In the commit graph, right-click the commit → *Revert* . Makes a new commit that reverses it. History stays intact.

Need one old file back

Open the file's *Timeline* (bottom of Explorer), pick the version, copy or restore.

Committed work is essentially impossible to lose. The only dangerous state is *uncommitted*.

An agent editing your project is a fast collaborator with no memory of yesterday. Git gives you the two things that make that safe: an undo button and a readable record.

BEFORE

Commit first, always

A clean commit before the agent starts means every change it makes is visible as a diff and reversible in one click.

DURING

Ask it to commit as it goes

“Commit after each working step with a clear message.” Small commits let you revert one step, not the whole session.

AFTER

Review the diff, not the file

Open Source Control and read what changed. Fluent code that looks right is exactly what the hallucination slide warned about.

CONTEXT

The history is a memory the agent can read

Diffs and commit messages tell an agent what was tried, what worked and why — cheaper and more precise than re-explaining.

Give it a branch. Let the agent work on *agent/fit-refactor* ; you review and merge into main like any pull request.

Never let it rewrite history. Commits and reverts only. If it asks to force-push or reset, say no.

Tests go in git too. Written before the work, committed, they are the check the agent cannot bend.

Start of day

Sync to pull anything new. Check you are on the right branch (bottom-left).

New idea

Click branch name → *Create new branch* . Name it for the idea.

While working

Edit → stage (+) → write message → *Commit* .
Every working step, many times a day.

Before an agent

Commit. Then let it work; ask it to commit as it goes.

Review

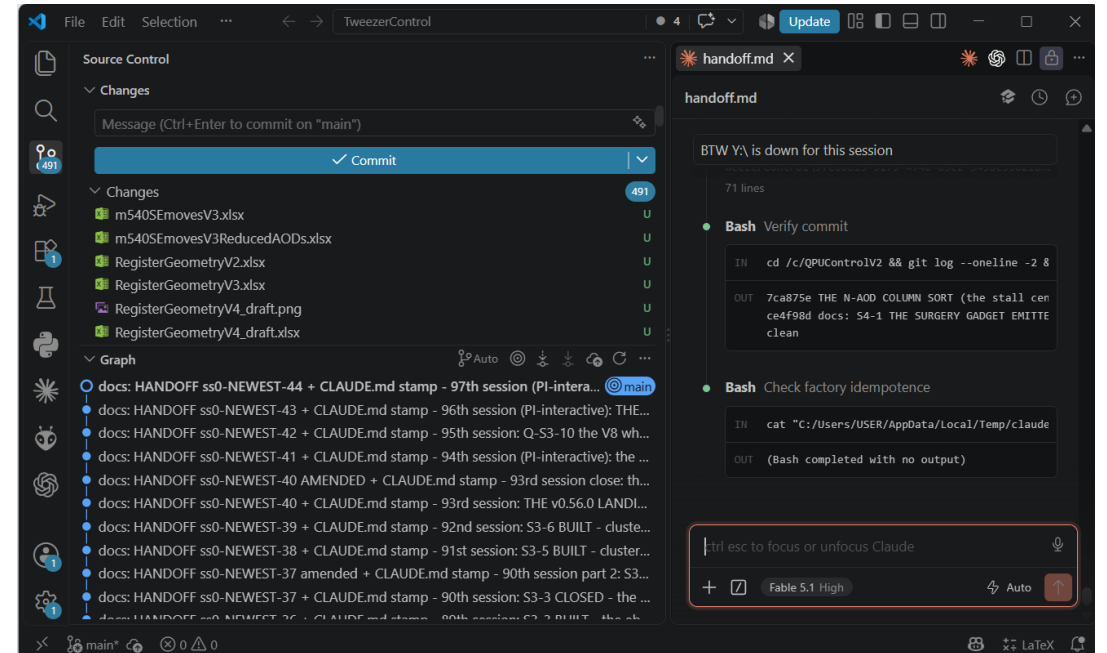
Click a changed file to see its diff. Discard or revert anything you don't want.

It works

Switch to main → *Branch* → *Merge* . Or push and open a pull request for review.

End of day

Sync so GitHub has everything. Laptops die; repositories don't.



SETUP, ONCE

Install Git (git-scm.com) and VS Code. Sign in to GitHub from VS Code's Accounts icon. Open any folder → Source Control → *Initialize Repository* .
Add a `.gitignore` before the first commit.

A SHORT PRIMER

Deployment: Free hosting platforms

How to get your work out to the world

Everything you build in this course can be shared with a URL at no cost. These are the services the lab actually uses.

CODE & COLLABORATION

GitHub

Repository hosting, issues, pull requests, and a wiki. Public repos are free and unlimited; private repos too. Add collaborators by username.

STATIC WEB PAGES

GitHub Pages

Any repo with an *index.html* becomes a website at *username.github.io/repo*. One setting, no server. Perfect for docs, reports, portfolios.

APPS WITH A BACKEND

Cloudflare Pages + Workers

Deploys straight from a GitHub repo on every push. Workers run server-side code (APIs, auth, scheduled jobs) on the free tier — ~100,000 requests a day.

VIDEO

YouTube (unlisted)

Upload once, share a link, embed anywhere. *Unlisted* keeps it off search but viewable by anyone with the URL — right for lab videos and talks.

DATA & NOTEBOOKS

Hugging Face · Zenodo

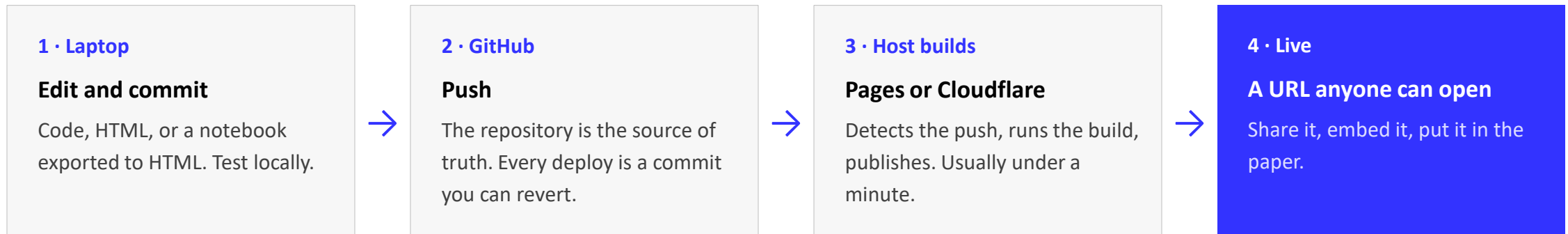
Hugging Face Spaces runs small Gradio/Streamlit demos free; Zenodo gives datasets a permanent DOI for citation.

ASK CLAUDE

“Deploy this”

Claude Code can set up the GitHub repo, enable Pages, write the Cloudflare config and push. You supply the accounts; it does the wiring.

You never upload files to a server. You **push to GitHub** ; the host watches the repository and redeploys automatically. Same loop for a one-page site or a full app.



Static vs. app. If the page only needs a browser — plots, text, a JavaScript calculator — GitHub Pages is enough. The moment it needs to store data, call an API with a secret key, or run on a schedule, use Cloudflare Workers.

Secrets never go in the repo. API keys live in the host's environment settings (Cloudflare → Settings → Variables). If a key is ever committed, revoke it — the history is public forever.

A SHORT PRIMER

Other ways to communicate and interact

LINE, Telegram, Gmail, building intelligent apps

A lab app is far more useful when it reaches your phone. All three follow the same shape: register a **bot**, get a **token**, run a tiny server that receives messages and sends replies.

LINE

LINE Messaging API

Create a channel in LINE Developers, get a channel token, set a webhook URL. Free tier: 200 push messages a month; replies to user messages are unlimited. What everyone in Taiwan already has.

TELEGRAM

Bot API via @BotFather

Message @BotFather, get a token in 30 seconds. Unlimited free messages, buttons, files, groups. The easiest place to start.

NOTIFICATIONS ONLY

ntfy.sh

No chat, just alerts. *ntfy.sh*: pick a topic name, subscribe in the app, send a POST — free, no account. “Run finished”, “temperature alarm”, “fit converged”.

WHAT TO TELL CLAUDE

```
“Build a Telegram bot on a Cloudflare Worker. When I send it a sample ID it replies with the latest measurement from my Google Sheet. Here is the bot token; store it as a secret. Also push a notification if any value exceeds 300 K.”
```

Webhook = the platform calls *your* URL whenever a message arrives. That URL is your Worker. No server to keep running.

Tokens are passwords. Paste them into the host’s secret store, never into code or chat history that gets committed.

Start with Telegram, then port to LINE — the code is nearly identical and Claude does the translation.

Do not paste bot token into claude chats! - secrets

Two routes, for two different situations: **you** working with Claude, or an **app** running on its own.

ROUTE A · YOU + CLAUDE, NO CODE

Connect Google to Claude and ask

Settings → Connectors → Google Drive, Gmail, Calendar. Then: “Summarise this week’s emails from the vendor”, “Put these fit results in a new Sheet”, “Draft the reply in Docs”.

Uses your login; works only while you are in the conversation. Right for one-off tasks and analysis.

ROUTE B · AN APP THAT RUNS ALONE

A service account, shared like a colleague

Google Cloud Console → create project → enable Sheets/Drive/Gmail API → create a *service account* → download its JSON key. Then share the Sheet or folder with the account’s email address.

Now your Worker or Python script can read and write that Sheet 24/7 without you. The JSON key is a secret — environment variable, never the repo.

Sheets as a database. For lab logs, sample lists and run tables a shared Sheet is the simplest store: humans edit it, code reads it, everyone sees the same thing.

Gmail for sending needs one extra step: domain-wide delegation or an app password. Claude will tell you which; for alerts, ntfy or Telegram is usually easier.

Say the route. “Set up a service account so this script can append to Sheet X” gets the right result; “connect to Google” is ambiguous.

Voice is not one thing — it is three pieces in a row, each of which you can rent as an API or run yourself.

1 · Speech → text

Transcribe

OpenAI Whisper (API or free local); browser *Web Speech API* for zero-cost in-page dictation; Telegram voice notes arrive as files you can transcribe.



2 · Think

The LLM call

Same API as the previous slide. Give it the transcript plus whatever the app knows — current readings, the run table — and ask for a short spoken-style answer.



3 · Text → speech

Speak

OpenAI TTS or ElevenLabs as APIs; *Chatterbox* (MIT, open source) runs on a lab GPU for free and clones a voice from a few seconds of audio.

Real-time voice APIs (OpenAI Realtime, Gemini Live) fold all three steps into one streaming connection with sub-second replies — right for a conversational assistant, overkill for logging.

Hands-free in the lab : a phone, a Telegram bot, voice notes. “Sample 14 loaded, pressure 2e-9.” The bot transcribes, parses, writes the Sheet, confirms by text. No custom hardware.

Latency adds up. Each step is a network round-trip. Keep spoken answers to a sentence or two and use the fast model tier.

A SHORT PRIMER

Agents: Giving software intelligence

API keys, agentic programming

Putting an LLM inside your app: API keys

Chat is you talking to a model. The **API** is your program talking to a model — a key, a request with text in, text out, billed per token. Claude and OpenAI work the same way.

WHAT A CALL LOOKS LIKE

```
response = client.messages.create( model="claude-sonnet-5",
system="You read spectrometer logs.", messages=[{"role":"user",
"content": log_text + "\n\nWhich runs failed and why?"}])
print(response.content[0].text)
```

Ten lines. Claude writes them; you supply the key as an environment variable.

Get a key

Anthropic Console (console.anthropic.com) or OpenAI Platform.

Pick a model

Cheap and fast for bulk (Haiku, GPT-mini); strong for judgement (Opus, GPT-5). Match the model to the step, as in the orchestration slides.

Watch cost

Tokens in + out. A one-page log through Haiku: a fraction of a cent; through Opus: a few cents. Set a monthly cap in the console.

Structured out

Ask for JSON — “return {run_id, status, reason}” — and your code can use it directly. Validate it; it is still a prediction.

Keys are money. Never in a repo, never in a client-side page anyone can view-source. Put them in the host’s secret store and call the model from the Worker, not the browser.

Same idea, other providers. Google Gemini, Mistral, local models via Ollama all expose a near-identical interface. Code written for one moves to another in minutes.

If you want to try we can provide API keys for testing

Keep these apart. One uses AI to **write** the program; the other puts AI **inside** the program. They have different costs and different powers.

A · AI-WRITTEN CODE

The model helps at development time. The program that ships contains no model.

Claude writes the fitting routine, the parser, the dashboard. You review, test, commit.

Runs anywhere, forever, for free. Deterministic: same input, same output. Fully testable.

Can only handle what was anticipated when it was written. New log format → it breaks.

Zero tokens at runtime.

B · LLM IN THE LOOP

The program calls the model while it runs. It can solve problems that did not exist at compile time.

“Here is an instrument log in a format I have never seen. Extract the failures.” It reads it anyway.

Handles free text, judgement, ambiguity, natural-language commands. Adapts without redeploying.

Costs tokens on every call, takes seconds, and is probabilistic — needs validation and fallbacks.

Tokens per call; intelligence on demand.

Default to A. If the task can be specified, have Claude write ordinary code for it. It is cheaper, faster and testable — and the model that wrote it is not needed to run it.

Reach for B at the hard step. Parsing messy text, deciding what to do next, talking to a human. Keep the LLM call small, give it structure to return, and check the result in ordinary code.

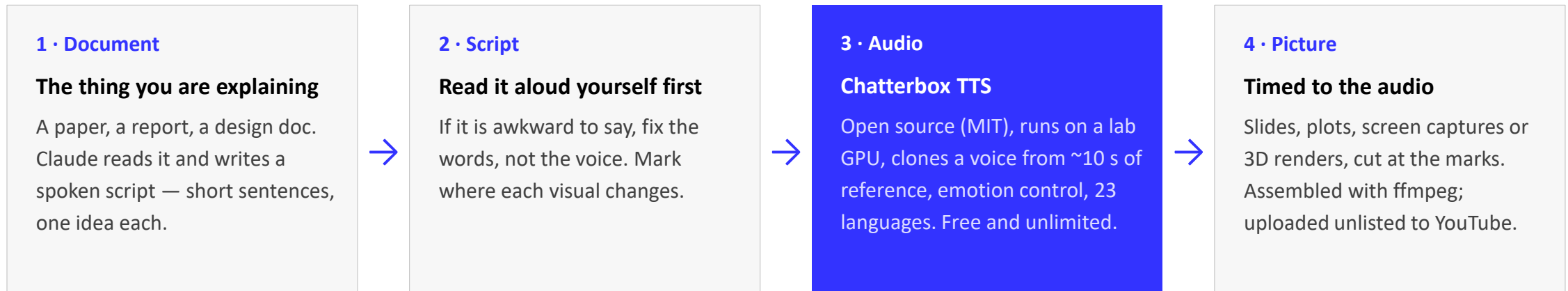
Agentic = B with tools. The model is allowed to call functions — query the Sheet, run the fit, send the message — and loop until done. Powerful; fence it with the checks from the verification slide.

A SHORT PRIMER

Video

How to get your work out to the world

An explainer video is a document read aloud with pictures timed to the words. Build it in that order — the picture is the *last* step, not the first.



The lab's pipeline is public: github.com/iams-yb-lab/show-your-work

Five Claude skills — *natural-voice*, *education-video*, *showoff-render*, *technical-report*, *slide-deck* — with the method and worked examples behind each. Open a Claude Code session in the repo and they load. Noncommercial licence; free for any academic lab.

Why audio before video. Speech sets the timing. Cut visuals to a finished narration and every transition lands on a word; do it the other way round and you re-record forever.

When the subject is a piece of apparatus, a render beats a photograph: no cables, any angle, exploded views, the parts appearing one by one. Two free tools, for two different jobs.

BLENDER · EFFICIENT, SCRIPTABLE

Clean renders and assembly animations from CAD

Import STEP/STL from Fusion, SolidWorks or KiCad. Studio lighting, materials, camera path, “parts fly in” — a few minutes of render on a desktop GPU.

Everything is Python underneath. Claude writes the script: “load board.step, explode along z, orbit 360° over 8 s, render 1080p”. Reproducible; re-run when the design changes.

The *showoff-render* skill in show-your-work is this workflow, with a worked example.

UNREAL ENGINE · REALISTIC SCENERY

Photoreal environments, real-time walkthroughs

When the *setting* matters: a full lab or beamline hall, a field station, a satellite in orbit. Real-time lighting and materials that hold up on a big screen.

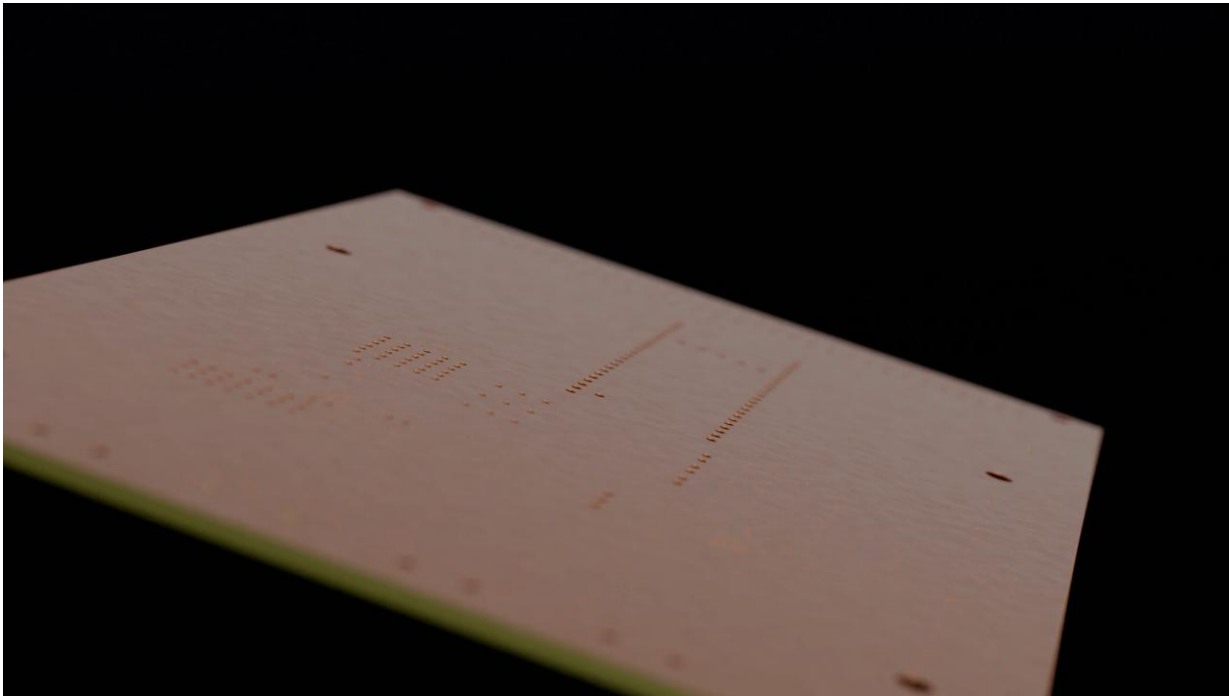
Free for non-commercial use; a large asset library. Heavier to learn and needs a serious GPU, but a walkthrough you can steer live is the payoff.

Import the Blender model of your instrument into the Unreal scene — the two tools compose.

Blender first. Nine times in ten the instrument is the point and Blender is enough. Reach for Unreal when the audience needs to feel the room.

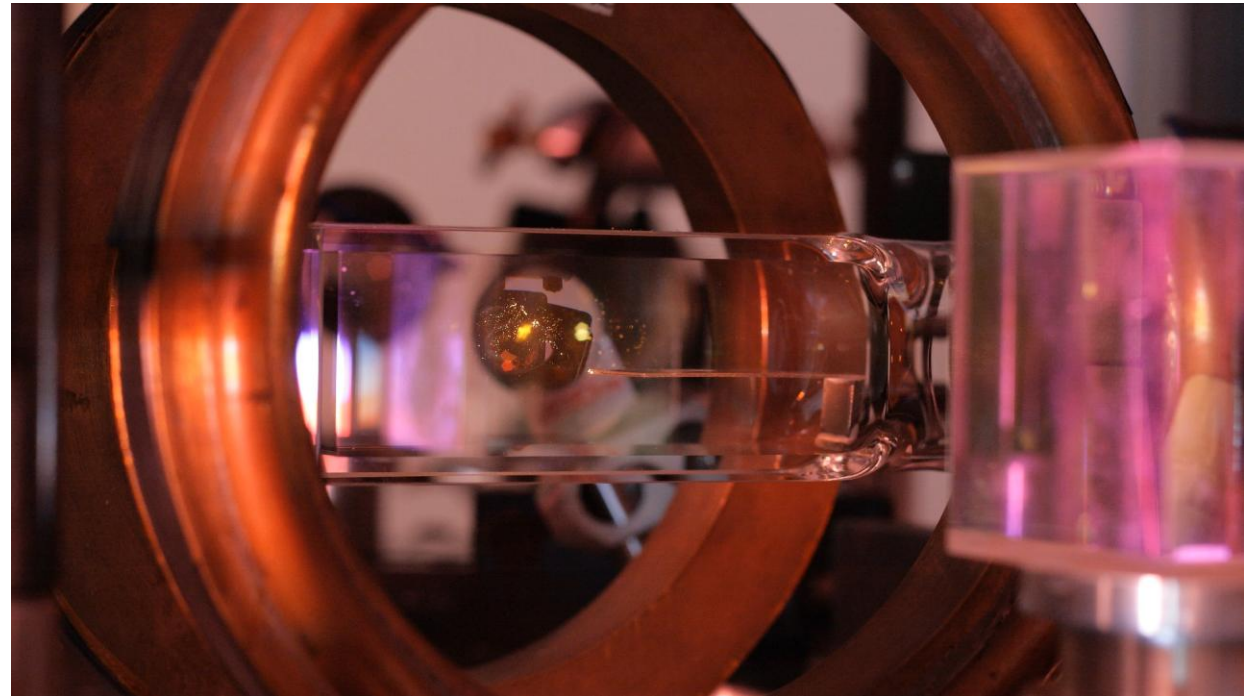
Render once, reuse. The same animation serves the talk, the paper’s graphical abstract, the group website and the funding report.

Label generated imagery. A render is not a photograph. Say so in the caption — the lab’s acceptable-use note is explicit about this.



<https://www.youtube.com/watch?v=5jy-V41uGpI>

<https://www.youtube.com/watch?v=mXi9sxOSgwc&t=11s>



<https://www.youtube.com/watch?v=dRVw6X3P0Is>

Let's Build...

1) What do you want to make?

- Suggestions: Loading a freight train game, Inverted pendulum simulator, phase locking coupled pendula, unfair golf, billiards, Calculator or simulator for something from your research?

2) Follow the “AI method” to produce a working local demo.

4) Deploy it on the web: github or cloudflare

5) Create website to explain the concept/game in detail and link to the live app and code.

6) Post your website to the class website / “project wall”

<https://tigp-experimental-methods.github.io/showcase-2026/>

7) Improve presentation: markdown -> slides -> website -> youtube video (optional)

1) Vision:

Describe what you want, with as much detail as possible,

2) Specification/Plan:

Get AI to produce a detailed specification - plan the project!

Break the project into many small tasks with testable deliverables

Ask AI to ask you questions!

3) Review:

Review the plan, have AI review the plan to identify improvements/flaws.

- eg Use Codex or a fresh session to review

4) Code (using agent swarms)

Tell AI to run swarms of agents in parallel to complete the build

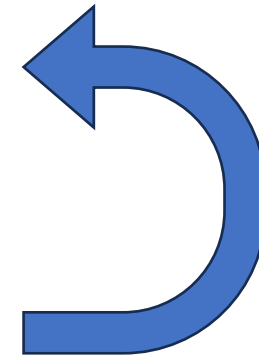
5) Test/Debug/Verify:

Ensure your plan/spec includes methods to test the code.

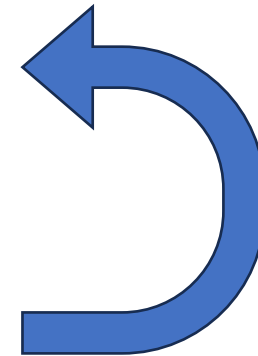
Fix problems one at a time

The most important slide in the talk!

Iterate the plan:
The first plan is *never* the last plan



Iterate



Iterate

***Never* trust AI without testing and verifying.**

Let's control a microcontroller ...over the net with our phone

- 1) Flash the class reference firmware first
(hardware baseline separates any hardware from software problems).
- 2) Create a phone app that allows you to control and measure from hardware (ESP32 – \$400NTD) over the internet (Wifi) (camera available ESP32-S3-N16R8-CAM + OV3660)

<https://jin-hua.com.tw/page/product/show.aspx?num=40729&kw=ESP32&lang=TW>

Eg: make a phone app to control the brightness and colour of the ESP32

- 3) Follow the “AI method”
- 4) Create website to explain the concept/game in detail and link to the live app and code.
- 5) Post your website to the class website / “project wall”

<https://tigp-experimental-methods.github.io/showcase-2026/>

- 6) Improve presentation: markdown -> slides -> website -> youtube video (optional)

1. Complete preparation
(KiCad 10 installed and opened + a JLCPCB account)
2. Improve your apps, build and have fun....